

Implementarea unui Procesor RISC-V cu Pipeline pe 5 Etape

Arhitectura Procesoarelor Moderne

Cuprins

1. Introducere
2. Arhitectura Procesorului RISC-V cu Pipeline
3. Etapa IF -- Instruction Fetch
4. Etapa ID -- Instruction Decode
5. Registrul de Pipeline IF/ID
6. Etapa EX -- Execute
7. Etapa MEM -- Memory Access
8. Etapa WB -- Write Back
9. Unitatea de Forwarding
10. Unitatea de Detectie a Hazardurilor
11. Registrele de Pipeline EX/MEM si MEM/WB
12. Microprocesorul RISC-V Complet
13. Bibliografie

Partea I

Front-end-ul Procesorului

(Etapele IF si ID)

Lucrarea 2

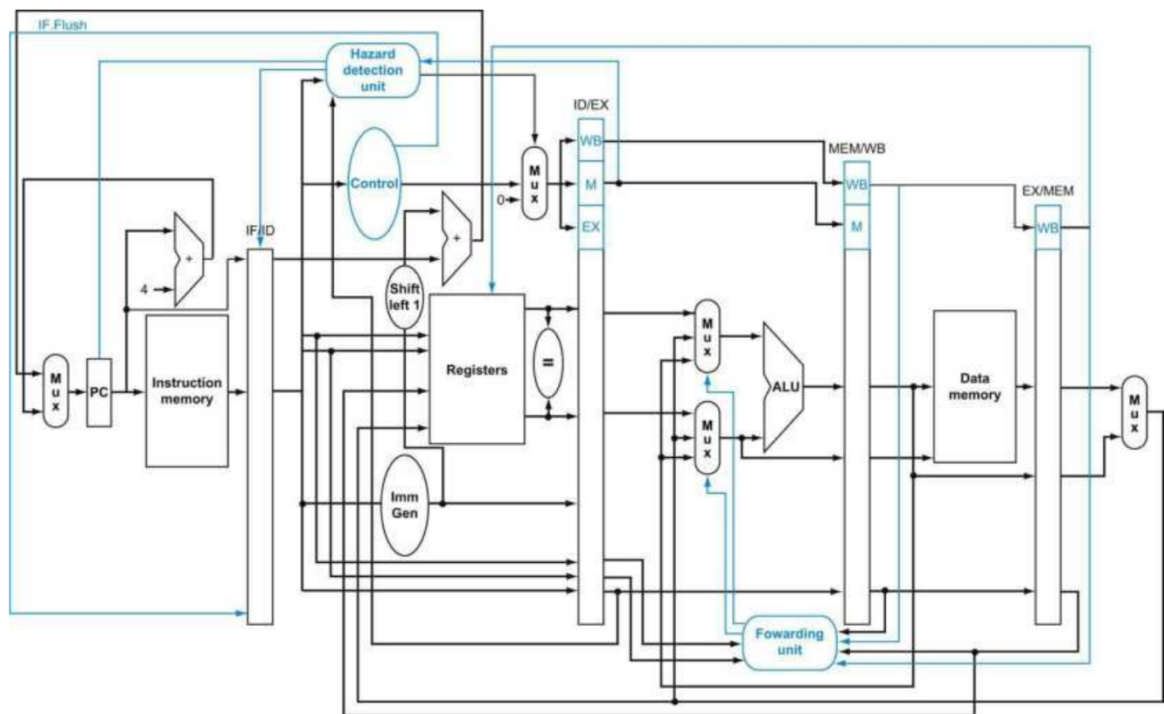


Fig. 1: Pipeline RISC-V

O arhitectură de set de instrucțiuni sau ISA (Instruction Set Architecture) este un model abstract al arhitecturii unui computer, definind lucruri precum modelul de registru și instrucțiunile codului mașinii. O realizare a unui ISA, cum ar fi o unitate centrală de procesare (CPU), se numește implementare. ISA utilizate pe scară largă includ x86 și ARM.

Pronunțat „risk-five”, RISC-V este un ISA bazat pe principiile computerului cu set de instrucțiuni reduse (RISC). Spre deosebire de majoritatea celorlalte modele ISA, acesta este furnizat sub o licență open source care nu necesită taxe pentru utilizare.

Pe baza acestui ISA [2] se pot propune diferite design-uri de procesor, dar toate trebuie să respecte formatul și modul de execuție a instrucțiunilor propuse. Procesorul ce va fi studiat și implementat în cadrul laboratorului va urma principiile expuse în [1] și [2], respectând extensia RV32I (pagina 9 din [2]) ce propune un procesor pe 32 de biți (atât pentru date cât și pentru instrucțiuni) capabil să execute instrucțiuni aritmetico-logice cu numere întregi fără suport hardware pentru virgulă mobilă sau operații de înmulțire/împărțire.

Fig. 1 prezintă schema bloc a unui procesor RISC-V cu 5 etape de pipeline:

1. IF(Instruction Fetch): citirea instrucțiunii curente
2. ID(Instruction Decode): decodarea instrucțiunii și citirea registrilor
3. EX(Execute): executarea instrucțiunii
4. MEM(Memory): operații de citire/scriere cu memoria de date
5. WB(Write Back): scrierea rezultatelor finale înapoi în registri

Lucrarea de fata isi propune implementarea a ceea ce se numește “front-end-ul” unui processor. Acesta se ocupă cu citirea și decodificarea instrucțiunilor ce urmează a fi executate de către processor și este reprezentat în acest design de către primele 2 etape de pipeline: IF+ID.

Pentru a executa o instructiune, primii pasi pe care trebuie sa-i facem, sunt sa citim instructiunea din memorie(mai precis codificarea acesteia), si sa o decodificam pentru a intelege anumite aspecte despre ea: ce tip de instructiune este, ce operatie trebuie executata, ce registri trebuie folositi, etc.

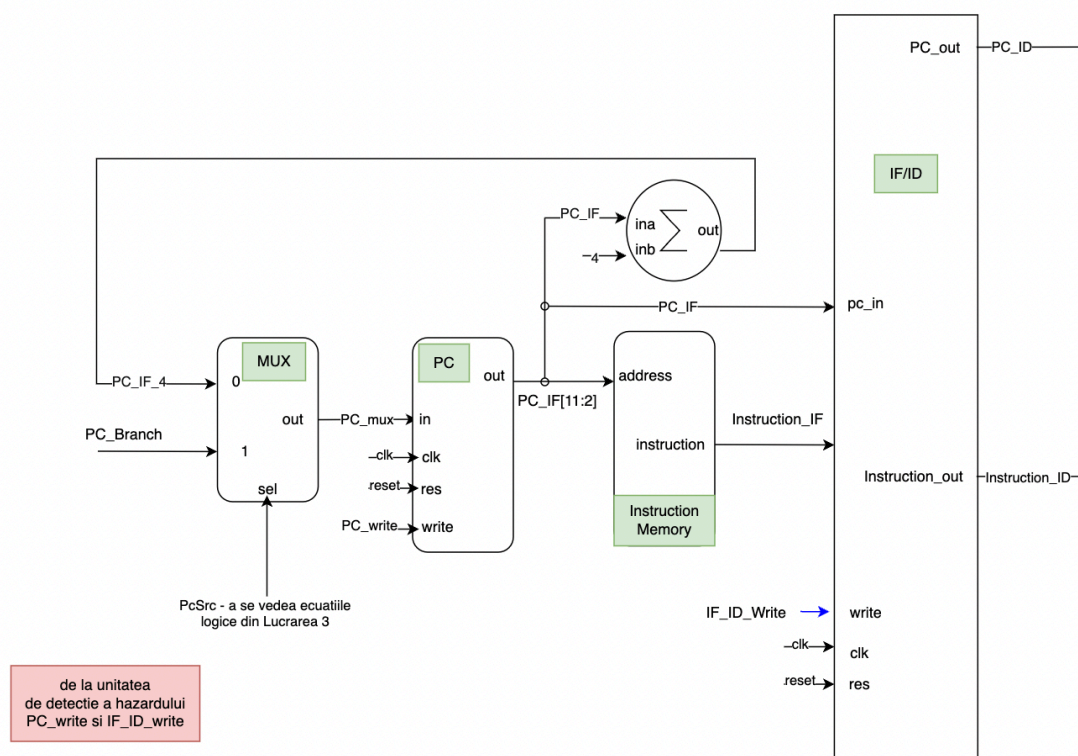


Fig. 2: Etapa IF împreună cu registrul de pipeline IF/ID

Pentru a ne pregati de executia urmatoarei instructiuni, trebuie de asemenea sa incrementam contorul de program(program counter - PC), astfel incat adresa sa de iesire sa reprezinte urmatoarea instructiune din memorie, aflata la o distanta de 4 octeti (32 de biți) de instructiunea curenta(mai multe detalii la Cap 4.3 din [1])

Această adresă este trimisă mai departe la memoria de instrucțiuni de unde este extrasă instrucțiunea corespundentă, instrucțiune ce este trimisă mai departe la etapa ID pentru decodificare printr-un registru de pipeline.

Întru-cât procesorul va fi implementat în două lucrări, etapele IF și ID vor conține în implementarea curentă semnale ce provin din etapele viitoare ce vor fi implementate în lucrarea 3. Din acest punct de vedere, pentru a putea implementa

complet front-end-ul procesorului în această etapa, vom simula în testbench acele semnale ce nu pot fi generate în lucrarea curentă.

Spre exemplu, în etapa IF, semnalele IF_ID_Write și PC_Write din Fig. 2 sunt generate de către unitatea de detecție a hazardurilor, unitate ce va fi implementată în Lucrarea 3. Din acest motiv, în implementarea front-end-ului noi vom implementa logica de funcționare pentru aceste semnale, iar pentru a testa front-end-ul, vom simula valorile acestor semnale în testbench. Acest lucru este valabil și pentru semnalele PC_Branch și PcSrc.

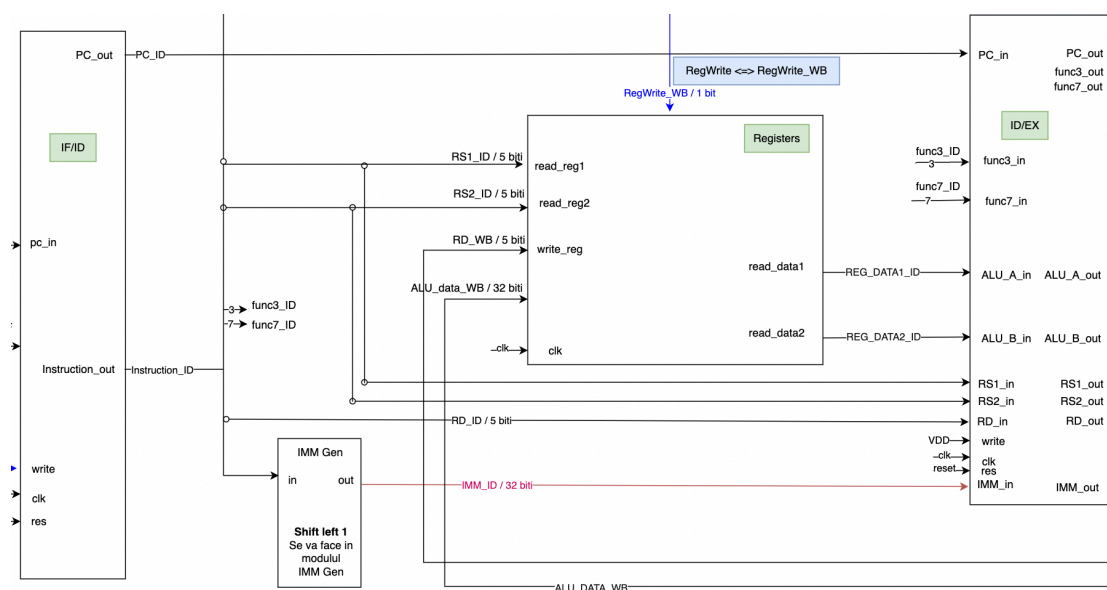


Fig. 3: Etapa ID împreună cu regiștrii de pipeline IF/ID și ID/EX

În etapa ID, instrucțiunea citită în IF este descompusă în mai multe câmpuri precum adresele regiștrilor sursă RS1 și RS2 (dacă este cazul), registrul destinație RD, valoarea imediată IMM (dacă este cazul) și diferite codificări pentru operația ce trebuie executată în câmpurile OPCODE, FUNCT3 și FUNCT7. Toate aceste câmpuri definesc ce face instrucțiunea, și ce regiștri sunt folosiți.

Și în cadrul acestei etape vom avea o serie de semnale ce vor trebui simulate din cauza faptului că acestea vor fi implementate propriu-zis în Lucrarea 3: RD_WB, RegWrite_WB, și ALU_DATA_WB.

Atât aceste semnale cât și cele de la etapa IF sunt toate simulate în cadrul fișierului de testbench furnizat a cărui simulare poate fi văzută în Fig. 4.

Cerințe:

1. Sa se implementeze urmatoarele module din IF:

a) Mux:

```
module mux2_1(input [31:0] ina, inb,
              input sel,
              output [31:0] out);
```

- b) PC – un registru de tipul D. Semnalul **clk** și semnalul **res** sunt semnale unice la nivelul pipe-ului.

```
module PC(input clk,res,write,
          input [31:0] in,
          output reg [31:0] out);
```

- c) Memorie Instrucțiuni - cu citire asincronă. În acest caz nu este nevoie de semnal **clk**. (hint: folosiți \$readmemh pt a încărca conținutul memoriei dintr-un fișier; pentru codificările instrucțiunilor și opcode-urile aferente, se va consulta documentul **riscv-spec-v2.2.pdf, paginile 104-107**):

```
module instruction_memory(input [9:0] address,
                          output reg [31:0] instruction);
```

Exemplu de folosire a directivei \$readmemh pentru încărcarea conținutului fișierului *code.mem* în memoria **codeMemory** (codul este folosit în modulul *instruction_memory*):

```
reg [31:0] codeMemory [0:1023];

initial $readmemh("code.mem", codeMemory);
```

De asemenea, în cazul de față cuvântul memoriei noastre este pe 32 de biți (o linie de memorie va cuprinde toți cei 32 de biți ai unei instrucțiuni). Procesorul nostru RISC-V știe că o instrucțiune este formată din 4 octeți, și astfel se așteaptă ca instrucțiunea următoare să se afle la 4 octeți de cea curentă (de aici și adunarea cu 4 a PC-ului).

O problemă ce apare în varianta propusă mai sus a memoriei, este că Verilog va accesa instrucțiunile din memorie la distanță de o linie, una de cealaltă (prima instrucțiune va fi pe prima linie, a doua instrucțiune pe cea de-a doua linie ș.a.m.d.).

Pentru a rezolva această problemă, va trebui să shiftăm adresa noastră PC la dreapta de două ori ($PC \gg 2$) înainte de a extrage ultimii 10 biți pentru adresa instrucțiunii noastre. Motivul este următorul: adresa noastră PC va fi un multiplu de 4, din cauza adunarilor repetate cu 4, astfel pentru a trece de la o secvență de adrese multiple de 4, la o secvență de adrese consecutive, cu distanță 1, vom face o împărțire la 4 (sau o shiftare la dreapta cu 2).

Obs: Cunoașteți o metodă în Verilog pentru a extrage direct cei 10 biți de adresă corecți fără a folosi nici măcar shiftarea?

- d) Sumator:

```
module adder(input [31:0] ina,inb,
             output reg [31:0] out);
```

2. Să se completeze modulul IF care conectează modulele definite la cerința 1 având următoarele intrări și ieșiri:

```
module IF(input clk, reset,
          input PCSrc, PC_write,
          input [31:0] PC_Branch,
          output [31:0] PC_IF, INSTRUCTION_IF);
```

3. Să se implementeze și simuleze separate bancul de registre (Registers) din ID, având următoarele intrări și ieșiri:

```

module registers(input clk,reg_write,
                 input [4:0] read_reg1,read_reg2,write_reg,
                 input [31:0] write_data,
                 output [31:0] read_data1,read_data2);

```

Hint: Bancul de registri pentru procesorul RISC-V contine 32 de registri pe 32 de biti, notati de la x0 la x31 (x0 va fi mereu 0). Bancul reprezinta o memorie multi-port cu 2 porturi de citire pentru citirea celor 2 operanzi si un port de scriere unde va fi scris registrul rezultat.

Semnalul sincron **reg_write** activ pe 1 logic, declanseaza scrierea in bancul de registri la adresa data de **write_reg** a valorii **write_data**.

Citirea este asincrona si se realizeaza imediat cu schimbarea adreselor de citire **read_reg1** respectiv **read_reg2**, valorile citite aflandu-se in **read_data1**, respectiv **read_data2**

Pentru o mai buna simulare, folositi un ciclu for(intr-un bloc initial definit in modulul registers) in care initializati fiecare registru cu numere de la 0 la 31.

4. Sa se implementeze unitatea de generare a datelor imediate cu urmatoarea definitie a modulului:

```

module imm_gen(input [31:0] in,
               output reg [31:0] out);

```

Generarea valorilor imediate se va face pentru urmatoarele instructiuni:

- lw
- addi
- andi
- ori
- xori
- slti
- sltiu
- srli,srai
- slli
- sw
- beq,bne,blt,bge,bltu,bgeu

Hint: urmariti generarea valorii imediate specificata in tabelul de mai jos, in functie de tipul instructiunii: I,S,B,U,J(se va consulta documentul **riscv-spec-v2.2.pdf, pagina 12, Fig 2.4**):

31	30	20	19	12	11	10	5	4	1	0	
— inst[31] —						inst[30:25]	inst[24:21]	inst[20]	I-immediate		
— inst[31] —						inst[30:25]	inst[11:8]	inst[7]	S-immediate		
— inst[31] —					inst[7]	inst[30:25]	inst[11:8]	0	B-immediate		
inst[31]	inst[30:20]		inst[19:12]		— 0 —						U-immediate
— inst[31] —			inst[19:12]	inst[20]	inst[30:25]	inst[24:21]	0	J-immediate			

5. Sa se completeze modulul ID care conecteaza modulele definite la cerințele 3 și 4 avand urmatoarele intrari si iesiri:

```
module ID(input clk,
  input [31:0] PC_ID, INSTRUCTION_ID,
  input RegWrite_WB,
  input [31:0] ALU_DATA_WB,
  input [4:0] RD_WB,
  output [31:0] IMM_ID,
  output [31:0] REG_DATA1_ID, REG_DATA2_ID,
  output [2:0] FUNCT3_ID,
  output [6:0] FUNCT7_ID,
  output [6:0] OPCODE_ID,
  output [4:0] RD_ID,
  output [4:0] RS1_ID,
  output [4:0] RS2_ID);
```

6. Sa se implementeze registrul de pipeline ce leaga IF de ID
7. Sa se implementeze si simuleze un modul top care sa uneasca cele 2 module IF si ID cu urmatoarele intrari si iesiri, folosind fisierul *ID.v* dat:

```
module RISC_V_IF_ID(input clk, //semnalul de ceas global
  input reset, //semnalul de reset global

  //semnale provenite din stagii viitoare
  //sunt pre-setate pentru aceasta lucrare
  //vor fi discutate in lucrarile urmatoare
  input IF_ID_write, //semnal de scriere pentru registrul de pipeline IF_ID
  input PCSrc_PC_write, //semnale de control pentru PC
  input [31:0] PC_Branch, //PC-ul calculat in etapa EX pentru instructiunile de salt
  input RegWrite_WB, //semnal de activare a scrierii in bancul de registri
  input [31:0] ALU_DATA_WB, //rezultatul calculat de ALU
  input [4:0] RD_WB, //registrul rezultat in care se face scrierea

  //semnale de iesire din ID
  //vor fi vizualizate pe simulare
  output [31:0] PC_ID, //adresa PC a instructiunii din etapa ID
  output [31:0] INSTRUCTION_ID, //instructiunea curenta in etapa ID
  output [31:0] IMM_ID, //valoarea calculata
  output [31:0] REG_DATA1_ID, //valoarea primului registru sursa citit
  output [31:0] REG_DATA2_ID, //valoarea celui de-al doilea registru sursa citit

  output [2:0] FUNCT3_ID, //funct3 din codificarea instructiunii
  output [6:0] FUNCT7_ID, //funct7 din codificarea instructiunii
  output [6:0] OPCODE_ID, //opcode-ul instructiunii
  output [4:0] RD_ID, //registru destinatie
  output [4:0] RS1_ID, //registru sursa1
  output [4:0] RS2_ID); //registru sursa2
```

8. Codificati instructiunile RISC-V urmatoare conform informatiilor din Fig. 3 (pentru codificarile instructiunilor si opcode-urile aferente, se va consulta documentul *riscv-spec-v2.2.pdf*, **paginile 104-107**):

```
add x2,x1,x0
addi x1,x1,1
and x3,x1,x2
ori x4,x1,1
sw x4,4(x5)
lw x12,8(x0)
beq x18,x0,5c
```

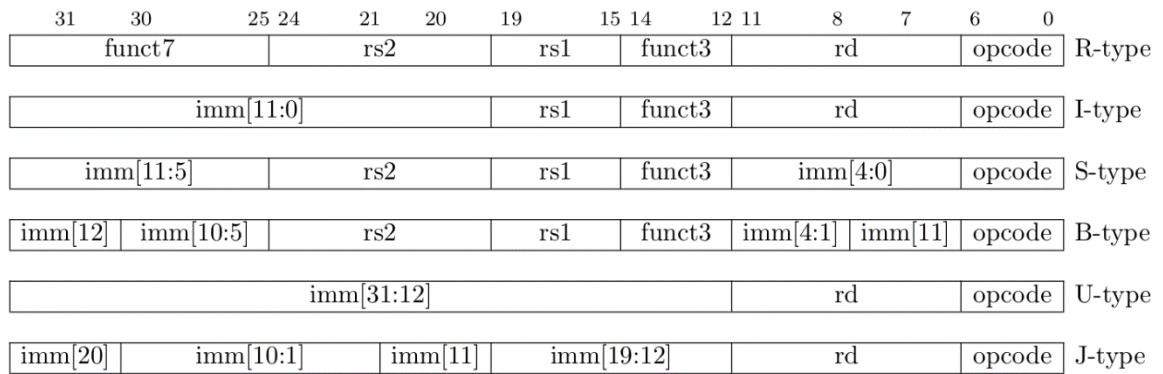


Fig. 3: Formatul instructiunilor aritmetice(R), imediate(I), salt conditionat(B), lucru cu memoria(S), salt neconditionat(J), upper-immediate(U) ale procesorului RISC-V

Pentru a verifica corectitudinea cerintelor, folositi fisierul de test oferit care ar trebui sa aiba urmatoarea simulare:

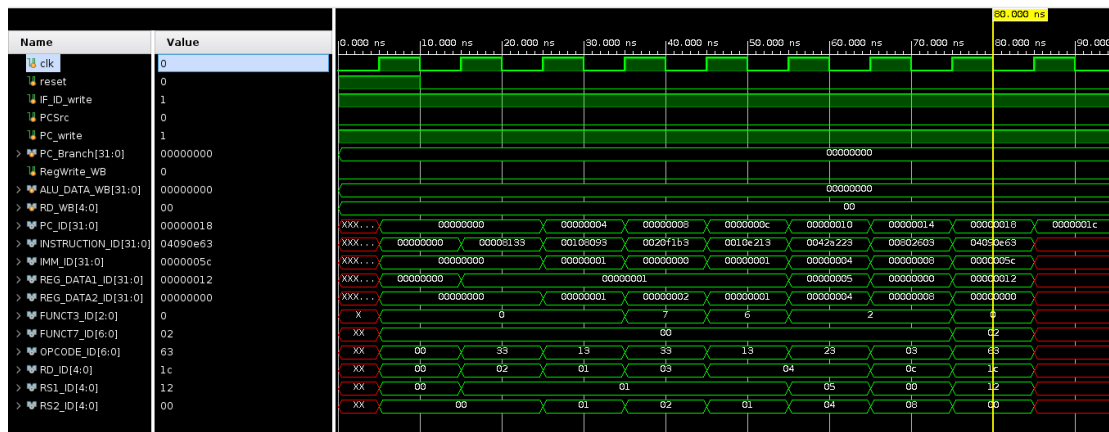


Fig. 4: Formele de unda rezultate in urma simularii intructiunilor specificate la cerinta 8, folosind fisierul de test dat

Bibliografie:

- [1] David A. Patterson, John L. Hennessy, *Computer Organization and Design RISC-V edition*, 2018
- [2] Andrew Waterman1, Krste Asanović, *The RISC-V Instruction Set Manual Volume I: User-Level ISA*, 2017

Partea II

Back-end-ul Procesorului

(Etapele EX, MEM, WB + Hazards)

Lucrarea 3

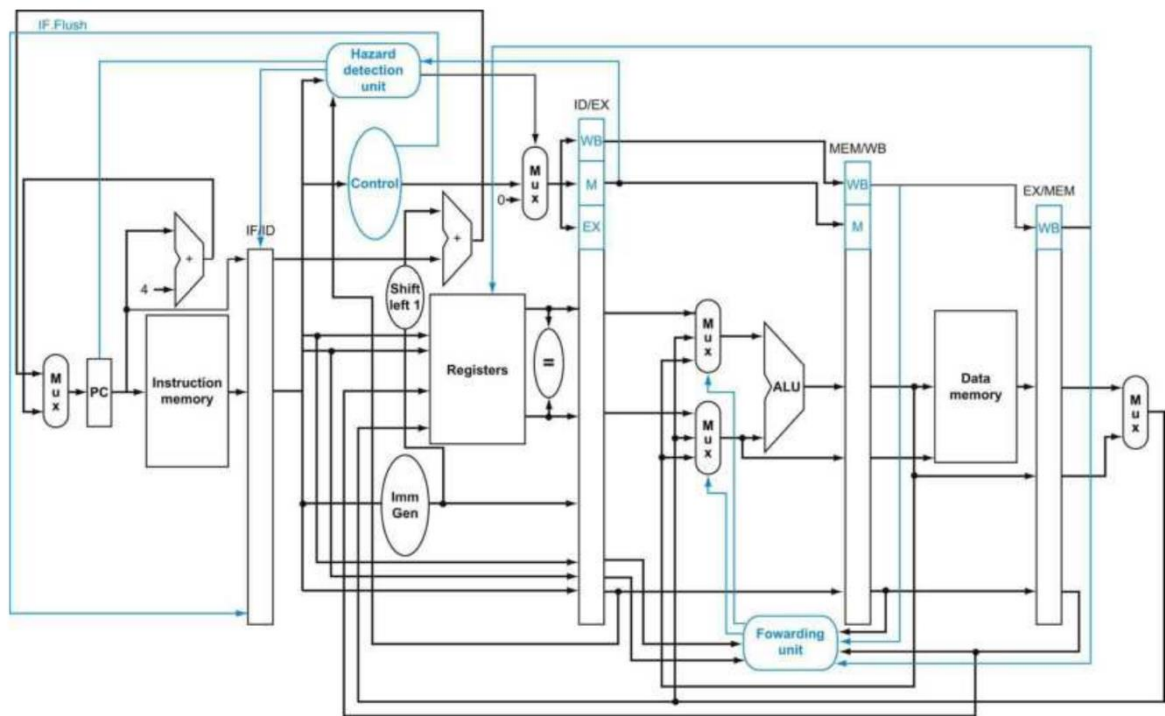


Fig. 1: Procesorul RISC-V – varianta pipeline

Lucrarea curenta isi propune implementarea etapelor EX, MEM, WB, plus doua unitati folosite in detectia hazardurilor si unitatea de forwarding, astfel realizandu-se un prototip complet a procesorului RISC-V.

Ca și element nou ce este oferit ca gata implementat pentru lucrarea curentă regăsim unitatea de control ce are rolul de a genera semnalele de control necesare fiecărei instrucțiuni în parte. Aceste semnale controlează logica diferitelor unități ale procesorului cum ar unitatea ALU, memoria de date, etc. De exemplu o instrucțiune logico-aritmetică trebuie să folosească unitatea ALU și să poată scrie rezultatul final înapoi în bancul de regiștri, în timp ce o instrucțiune de lucru cu memoria trebuie să poată controla memoria de date.

Din arhiva oferită pentru soluția lucrării anterioare, se poate folosi proiectul Vivado oferit direct, iar cerințele lucrării curente să fie implementate direct în acest proiect. În cazul în care se optează pentru a folosi propriul vostru proiect realizat pentru lucrarea anterioară, atunci va trebui să importați unitatea de control cât și modulul ID corespondent unității ID.

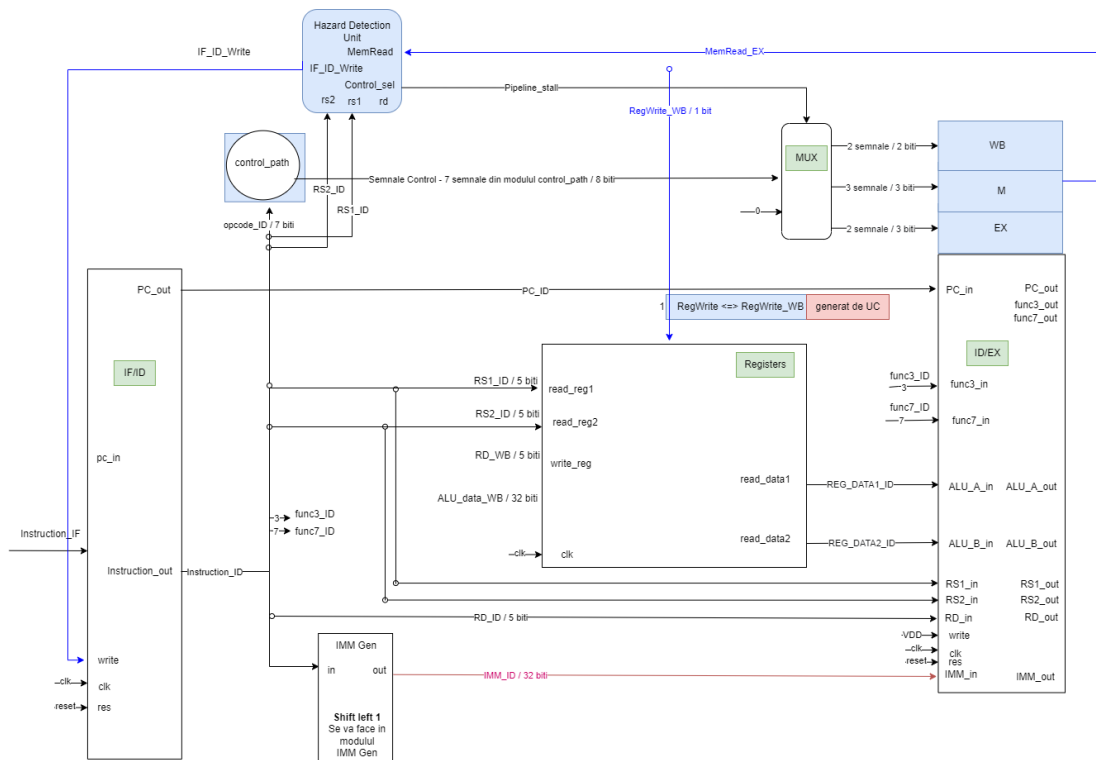


Fig. 2: Etapa ID complete cu unitatea de control (control_path) și unitatea de detecție a hazardurilor (hazard detection unit).

În etapa EX, unitatea ALU se va ocupa de executarea propriu-zisă a operației aritmetico-logice din instrucțiunea curentă. Un prototip minimal de ALU este descris prin tabelele din Fig. 3, unde în funcție de funct3, funct7 și AluOp (generat de unitatea de control), se generează semnalele de control corespunzătoare operației dorite.

ALU control lines	Function
0000	AND
0001	OR
0010	add
0110	subtract

Instruction opcode	ALUOp	Operation	Funct7 field	Funct3 field	Desired ALU action	ALU control input
ld	00	load doubleword	XXXXXX	XXX	add	0010
sd	00	store doubleword	XXXXXX	XXX	add	0010
beq	01	branch if equal	XXXXXX	XXX	subtract	0110
R-type	10	add	000000	000	add	0010
R-type	10	sub	010000	000	subtract	0110
R-type	10	and	000000	111	AND	0000
R-type	10	or	000000	110	OR	0001

Fig. 3: Tabelul logic pentru generarea semnalelor de control a unității ALU pentru instrucțiunile AND, OR, ADD, SUB

Etapa MEM se ocupa de scrierea/citirea în memoria de date de 4KB, similară cu cea de instrucțiuni. Aceasta suportă, pe lângă operația de citire asincronă, operația de scriere sincronă.

Etapa WB se ocupa de scrierea rezultatului final in bancul de registri. Rezultatul final este ales de un multiplexor, dintre valoarea citita din memoria de date(daca avem de a face cu instructiunea LW), sau valoarea calculata de ALU(daca avem de a face cu instructiune de tipul R-type sau I-type). Semnalul de control MemtoReg functioneaza ca semnal de selectie pentru multiplexorul mentionat.

Unitatea de Forwarding este responsabila pentru rezolvarea hazardurilor de tip RAW. Aceasta face bypass la valorile calculate de ALU ce nu au fost inca scrise in bancul de registri, si se afla inca in pipeline(in etapele MEM sau WB). Aceasta este prezentata in Fig. 4, si genereaza semnalele ForwardA si ForwardB ce sunt responsabile pentru alegerea operanzilor trimisi spre ALU (mai multe detalii în [1] de la pagina 569).

Valorile ce sunt folosite pentru forwarding sunt preluate din etapele viitoare, mai precis rezultatele produse de ALU sau citite din memoria de date ce au ajuns fie în etapa MEM sau WB.

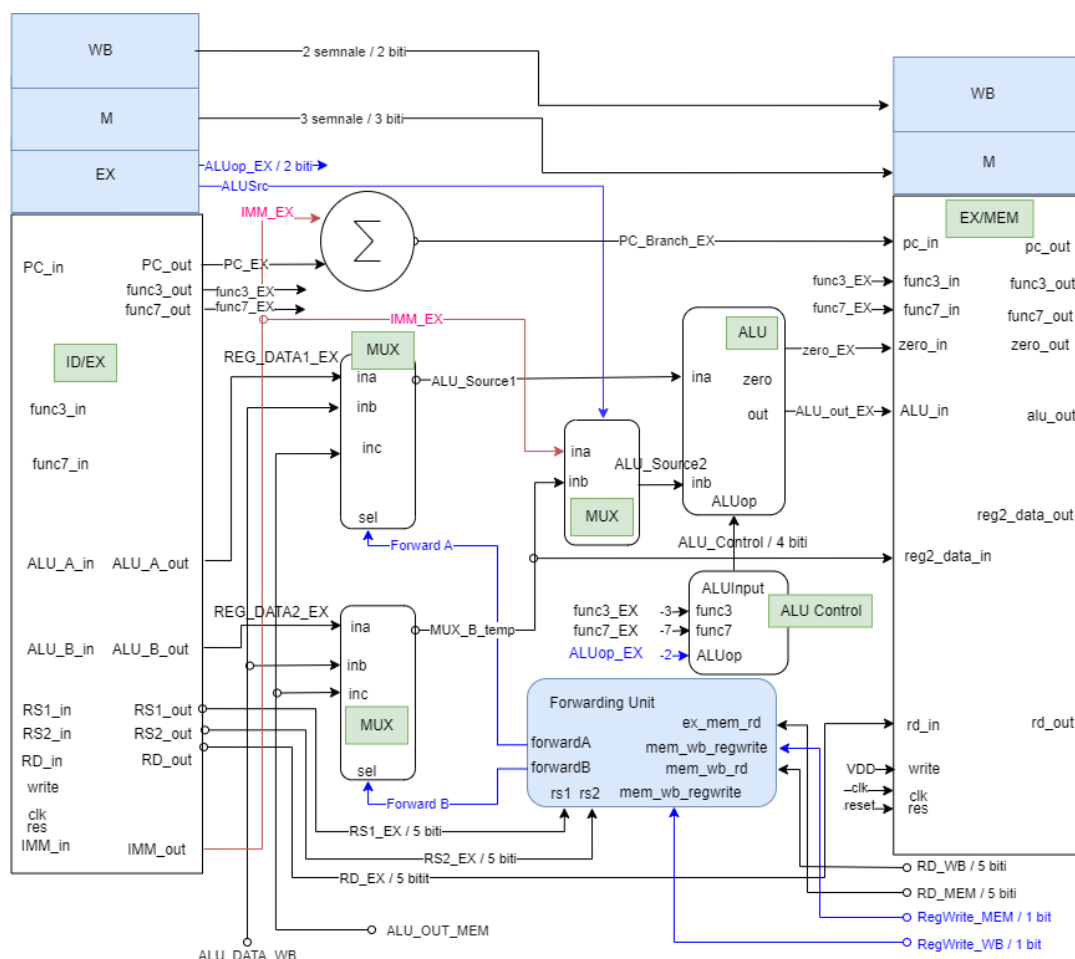


Fig. 4: Etapa EX completă cu Unitatea de Forwarding

Multiplexoarele din Fig. 4 ale căror semnale de control sunt Forward A și Forward B selectează valorile registrilor sursă ce vor fi trimiși spre ALU(fie aceștia din etapa ID, fie bypass-ate din MEM/WB). Multiplexorul ALU_Source2 selectează

valoarea de input pentru ALU, dintre o valoare imediata sau un registru. In Fig. 4 avem logica pentru a putea realiza operatii atat intre 2 registri, cat si intre un registru si o valoare imediata.

Pentru a completa etapa EX, este necesara o logica ce calculeaza adresa de salt pentru instructiunile de tip branch. Aceasta adresa este calculata prin adunarea adresei PC curente a instructiunii, si valoarea imediata codificata in aceasta. Fig. 5 prezinta aceasta logica unde valoarea PC cât și valoarea imediată ajunse în etapa EX sunt trimise într-un sumator separat unde este calculată adresa finală de salt. Aceasta este propagată mai departe în etapa MEM de unde va fi trimisă direct către etapa IF pentru a face saltul în funcție de valoarea semnalului PCSrc.

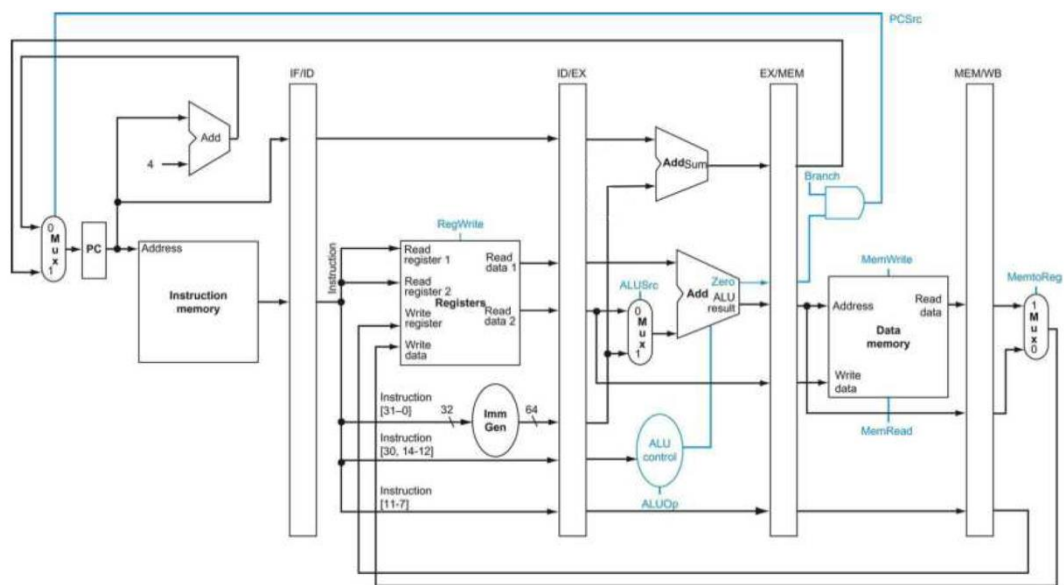


Fig. 5: Logica pentru calculul adresei de salt adaugata in etapa EX, al carui rezultat este propagat in etapa MEM. Aceasta este o figură simplificată pentru a putea vizualiza mai ușor traseul datelor. Etapa EX completă poate fi văzută în Fig. 4

Semnalul PCSrc se calculează în modulul Branch Control în funcție de tipul instrucțiunii și de rezultatul operației corespunzătoare efectuate în ALU pentru o instrucțiune de Branch (semnalul de control `BRANCH_MEM == 1`) după cum urmează:

Instrucțiune	FUNCT3_MEM	ALU_OUT_MEM[0]	ZERO_MEM	PCSrc
beq	000	X	1	1
bne	001	X	0	1
blt	100	1	X	1
bge	101	0	X	1
bltu	110	1	X	1
bgeu	111	0	X	1
în rest				0

Unitatea de detectie a hazardurilor (Hazard detection unit) insereaza stall-uri in pipeline atunci cand hazardul RAW nu poate fi rezolvat cu forwarding(mai multe detalii în [1] de la pagina 582). Aceasta poate fi revazută în Fig. 2 alături de etapa ID.

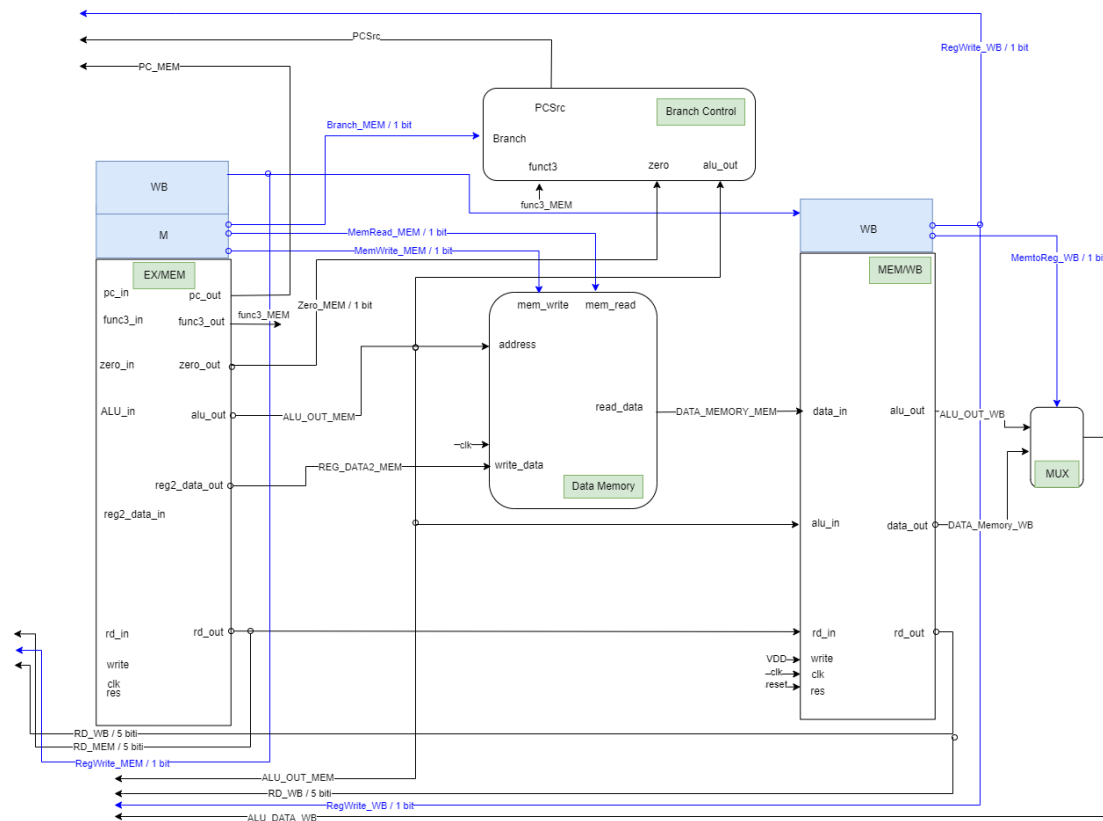


Fig. 6: Etapele MEM și WB

Cerințe:

1. Sa se implementeze unitatea ALUcontrol:

```
module ALUcontrol(input [1:0] ALUop,
                  input [6:0] funct7,
                  input [2:0] funct3,
                  output reg [3:0] ALUinput);
```

Semnalul ALUinput pe 4 biti va desemna operatia executata de ALU, in functie de instructiunea curenta (se vor folosi field-urile funct3 si funct7 si semnalul de control ALUop pentru a identifica ce operatie trebuie executata).

Se va folosi urmatorul tabel pentru generarea lui ALUinput in functie de instructiune:

Instruction	ALUinput code
ld, sd (w)	0010
add (i)	0010
sub (i)	0110
and (i)	0000
or (i)	0001
xor (i)	0011
srl (i)	0101
sll (i)	0100

sra (i)	1001
sltu	0111
slt	1000
beq, bne	0110
blt, bge	1000
bltu, bgeu	0111

2. Sa se implementeze unitatea ALU:

```
module ALU(input [3:0] ALUop,      //ALUinput ce selecteaza operatia
           input [31:0] ina,inb,  //operanzii ce iau parte la operatie
           output zero,           //semnal ce verifica daca rezultatul este 0
           output reg [31:0] out); //rezultatul operatiei
```

3. Sa se implementeze modulul EX ce cuprinde unitatea ALU, ALUcontrol, multiplexoarele necesare pentru alegerea operanzilor si adder-ul ce calculeaza adresa de salt:

```
module EX(input [31:0] IMM_EX,      //valoarea imediata in EX
           input [31:0] REG_DATA1_EX, //valoarea registrului sursa 1
           input [31:0] REG_DATA2_EX, //valoarea registrului sursa 2
           input [31:0] PC_EX,      //adresa instructiunii curente in EX
           input [2:0] FUNCT3_EX,   //funct3 pentru instructiunea din EX
           input [6:0] FUNCT7_EX,   //funct7 pentru instructiunea din EX
           input [4:0] RD_EX,       //adresa registrului destinatie
           input [4:0] RS1_EX,      //adresa registrului sursa 1
           input [4:0] RS2_EX,      //adresa registrului sursa 2
           input RegWrite_EX,       //semnal de scriere in bancul de registri
           input MemtoReg_EX,       //...
           input MemRead_EX,        //semnal pentru activarea citirii din memorie
           input MemWrite_EX,       //semnal pentru activarea scrierii in memorie
           input [1:0] ALUop_EX,    //semnalul de control ALUop
           input ALUSrc_EX,         //semnal de selectie intre RS2 si valoarea imediata
           input Branch_EX,         //semnal de identificare a instructiunilor de tip branch
           input [1:0] forwardA,forwardB, //semnalele de selectie pentru multiplexoarele de forwarding

           input [31:0] ALU_DATA_WB, //valoarea calculata de ALU, prezenta in WB
           input [31:0] ALU_OUT_MEM, //valoarea calculata de ALU, prezenta in MEM

           output ZERO_EX,           //flag-ul ZERO calculat de ALU
           output [31:0] ALU_OUT_EX, //rezultatul calculat de ALU in EX
           output [31:0] PC_Branch_EX, //adresa de salt calculata in EX
           output [31:0] REG_DATA2_EX_FINAL //valoarea registrului sursa 2 selectata dintre
           );                        //valorile prezente in etapele EX, MEM si WB
```

4. Sa se implementeze memoria de date din etapa MEM:

```
module data_memory(input clk,
                   input mem_read, //semnal de activare a citirii din memorie
                   input mem_write, //semnal de activare a scrierii in memorie
                   input [31:0] address, //adresa de scriere/citire
                   input [31:0] write_data, //valoarea scrisa in memorie
                   output reg [31:0] read_data //valoarea citita din memorie
                   );
```

5. Sa se implementeze unitatea de detectie a hazardurilor:

```

module hazard_detection(input [4:0] rd, //adresa registrului destinatie in etapa EX
                        input [4:0] rs1, //adresa registrului sursa 1 decodificat in etapa ID
                        input [4:0] rs2, //adresa registrului sursa 2 decodificat in etapa ID
                        input MemRead, //semnalul de control MemRead din etapa EX
                        output reg PCwrite, //semnalul PCwrite ce controleaza scrierea in registrul PC
                        output reg IF_IDwrite, //semnal ce controleaza scrierea in registrul de pipeline IF_ID
                        output reg control_sel); //semnal transmis spre unitatea de control

```

Unitatea de detectie a hazardurilor va fi implementata dupa urmatoarea logica:

```

if (ID/EX.MemRead and
    ((ID/EX.RegisterRd = IF/ID.RegisterRs1) or
     (ID/EX.RegisterRd = IF/ID.RegisterRs2)))
    stall the pipeline

```

Semnalele de output vor fi puse in locul liniei “stall the pipeline” astfel incat in momentul inserarii unui stall, registrul PC din IF si registrul de pipeline IF_ID sa nu mai permita actualizarea valorilor interne pana cand hazardul nu este rezolvat, iar semnalul control_sel va seta semnalele de control generate de unitatea de control pe 0 logic.

6. Sa se implementeze unitatea de forwarding:

```

module forwarding(input [4:0] rs1, //adresa registrului sursa 1 in etapa EX
                  input [4:0] rs2, //adresa registrului sursa 2 in etapa EX
                  input [4:0] ex_mem_rd, //adresa registrului destinatie in etapa MEM
                  input [4:0] mem_wb_rd, //adresa registrului destinatie in etapa WB
                  input ex_mem_regwrite, //semnalul de control RegWrite in etapa MEM
                  input mem_wb_regwrite, //semnalul de control RegWrite in etapa WB
                  output reg [1:0] forwardA,forwardB); //semnalele de selectie a multiplexoarelor
//ce vor alege valoarea ce trebuie bypassata

```

Unitatea de forwarding va fi implementat dupa urmatoarea logica:

1. Pentru hazard in etapa EX:

```

if (EX/MEM.RegWrite
    and (EX/MEM.RegisterRd ≠ 0)
    and (EX/MEM.RegisterRd = ID/EX.RegisterRs1)) ForwardA = 10
if (EX/MEM.RegWrite
    and (EX/MEM.RegisterRd ≠ 0)
    and (EX/MEM.RegisterRd = ID/EX.RegisterRs2)) ForwardB = 10

```

2. Pentru hazard in etapa MEM:

```

if (MEM/WB.RegWrite
    and (MEM/WB.RegisterRd ≠ 0)
    and not (EX/MEM.RegWrite and (EX/MEM.RegisterRd ≠ 0)
             and (EX/MEM.RegisterRd = ID/EX.RegisterRs1))
    and (MEM/WB.RegisterRd = ID/EX.RegisterRs1)) ForwardA = 01
if (MEM/WB.RegWrite
    and (MEM/WB.RegisterRd ≠ 0)
    and not (EX/MEM.RegWrite and (EX/MEM.RegisterRd ≠ 0)
             and (EX/MEM.RegisterRd = ID/EX.RegisterRs2))
    and (MEM/WB.RegisterRd = ID/EX.RegisterRs2)) ForwardB = 01

```

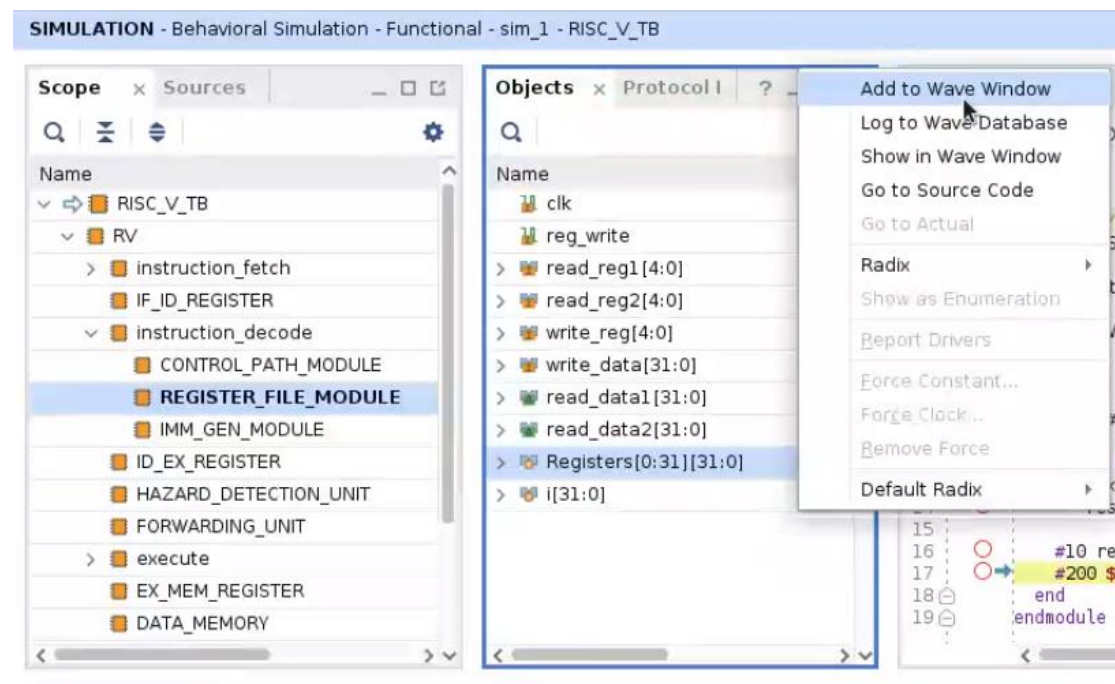
7. Sa se implementeze registrele de pipeline dintre etapele EX si MEM, respectiv MEM si WB

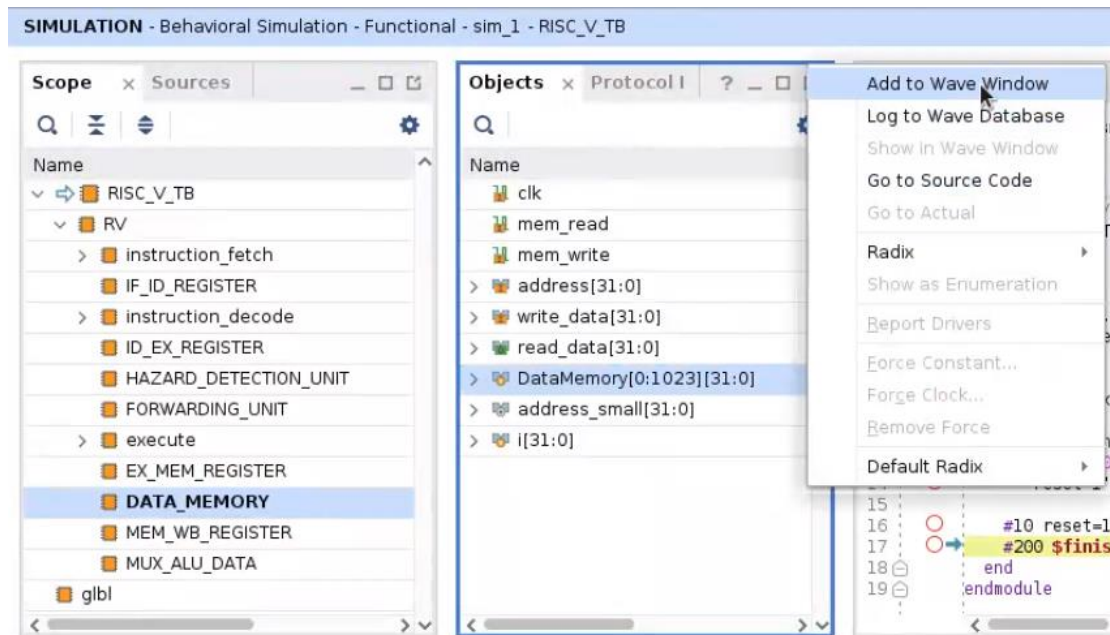
8. Sa se implementeze microprocesorul RISC-V complet ce contine cele 5 stagii de pipeline si cele 2 module pentru detectia hazardurilor, respectiv unitatea de forwarding:

```
module RISC_V(input clk, //semnalul de ceas global
             input reset, //semnalul de reset global

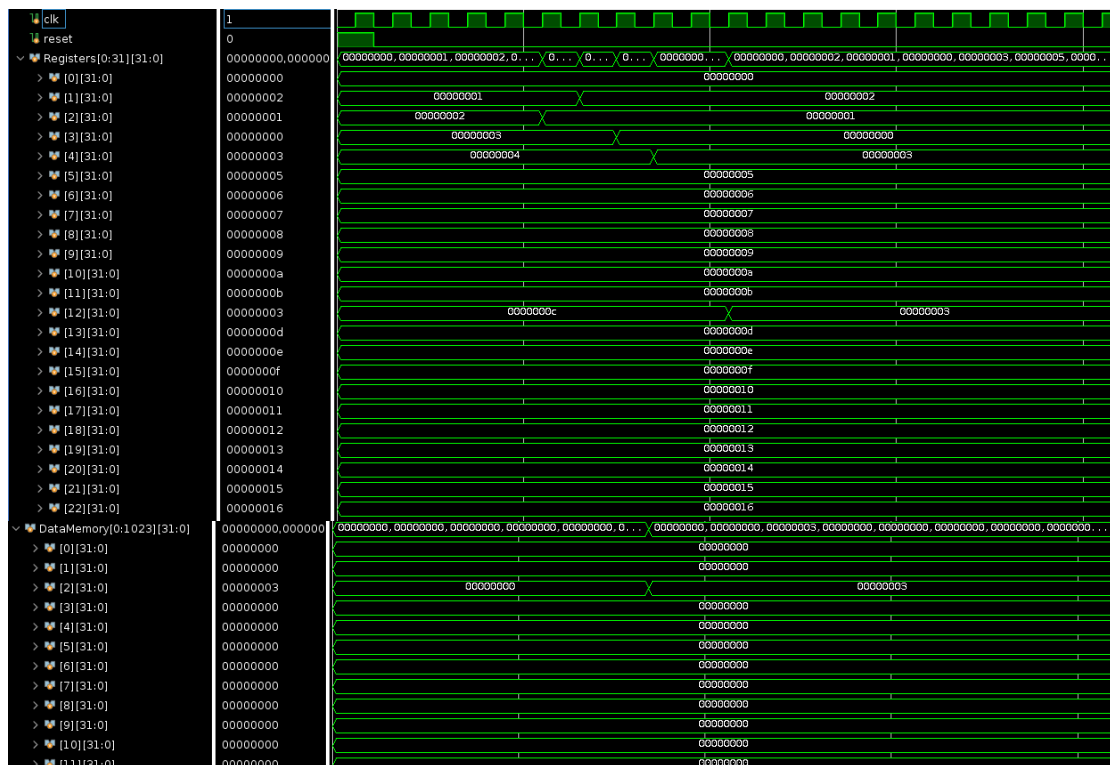
             output [31:0] PC_EX, //adresa PC in etapa EX
             output [31:0] ALU_OUT_EX, //valoarea calculata de ALE in etapa EX
             output [31:0] PC_MEM, //adresa de salt calculata
             output PCSrc, //semnal de selectie pentru PC
             output [31:0] DATA_MEMORY_MEM, //valoarea citita din memoria de date in MEM
             output [31:0] ALU_DATA_WB, //valoarea finala scrisa in etapa WB
             output [1:0] forwardA, forwardB, //semnalele de forwarding
             output pipeline_stall //semnal de stall la detectia de hazarduri
);
```

Pentru simulare se va adauga bancul de registri si memoria de date in fereastra pentru formele de unda in urmatorul fel:

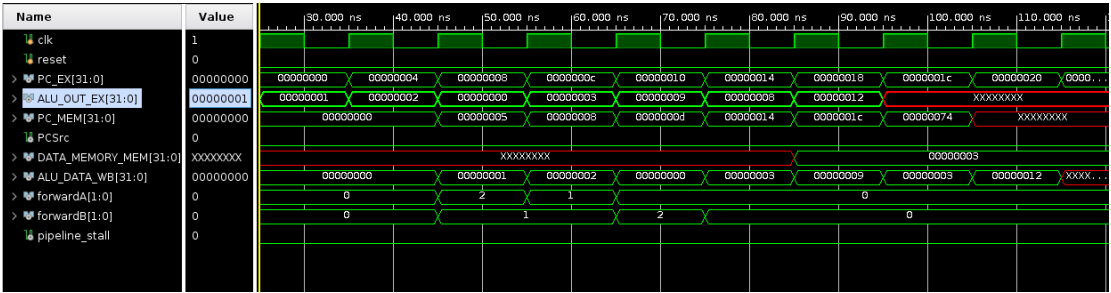




Simularea se va face pe baza codului assembly folosit in lucrarea precedenta, iar valorile finale rezultate in bancul de registri, respectiv memoria de date vor fi urmatoarele:



Simularea completa a modului RISC-V:9



Bibliografie:

[1] David A. Patterson, John L. Hennessy, *Computer Organization and Design RISC-V edition*, 2018